Lab Task on Python List Processing

Instruction: Use only core Python list operations. **Do not use pandas, numpy, or any external libraries.**

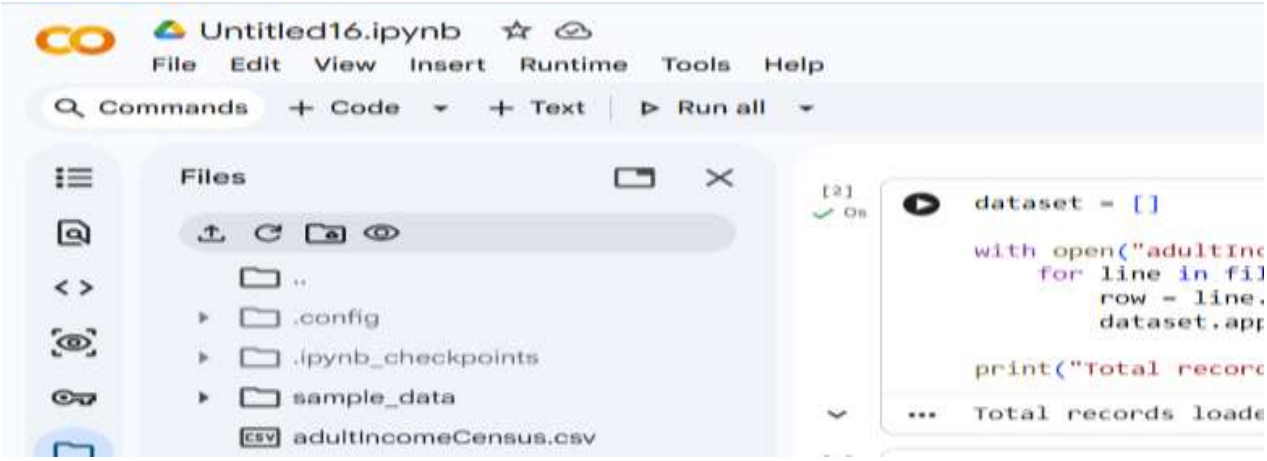
Task list:

After loading the dataset, complete the below tasks:

1. Print the details of the 10th record and the 19th record from the last. Use basic and negative indexing.
2. Find how many attributes each record contains.
3. Create separate lists for: Ages, Education levels, Occupations, Hours-per-week
4. Remove all records that contain missing values
5. Create: A subset containing the first 500 records and another one with the last 200 records.
6. Concatenate these two lists into a new list.
7. Replace all "<=50K" income labels with "Low" and ">50K" with "High".
8. Take the first 100 cleaned records and replicate them 5 times using *.
9. Select any one record and assign its fields into multiple variables using list unpacking.
10. Ask the user to input an occupation and print all matching records using membership checking.

Solution:

Uploading the dataset: (adultIncomeCensus.csv)



Problem 1: Print the details of the 10th record and the 19th record from the last. Use basic and negative indexing.

Code:

```
dataset = []
with open("adultIncomeCensus.csv", "r") as file:
for line in file:
row = line.strip().split(",")
dataset.append(row)
print("Dataset loaded. Total records:", len(dataset))

print("10th record:")
print(dataset[9])
print("19th record from last:")
print(dataset[-19])
```

Output:

Explanation:

Colab read the CSV file line by line and store each record as a **list inside another list** . Python lists use **0-based indexing**.
So the 10th record is index **9**.
Negative indexing counts from the end.

Problem 2: Find how many attributes each record contains.

Code:

```
attributes = len(dataset[0])
print("Number of attributes per record:", attributes)
```

Output:

```
[4]
✓ 0s
attributes = len(dataset[0])
print("Number of attributes per record:", attributes)

Number of attributes per record: 15
```

Explanation: Each record in the dataset is stored as a list. The number of attributes equals the length of one record. The len() function is used to count elements inside a list.We check the first record to determine attribute count.

Problem 3: Create separate lists for: Ages, Education levels, Occupations, Hours-per-week.

Code:

```
ages = []
education_levels = []
occupations = []
hours_per_week = []
ages = []

education_levels = []

occupations = []

hours_list = []

for row in dataset:

    ages.append(row[age])

    education_levels.append(row[education])

    occupations.append(row[occupation])

    hours_list.append(row[hours_per_week])

print("Lists created successfully!")
```

Output:

```
[9]
✓ 0s
ages = []
education_levels = []
occupations = []
hours_list = []

for row in dataset:
    ages.append(row[age])
    education_levels.append(row[education])
    occupations.append(row[occupation])
    hours_list.append(row[hours_per_week])

print("Lists created successfully!")

Lists created successfully!
```

Explanation:

We extract specific columns using their **index positions**:

Field	Index
Age	0
Education	3
Occupation	6
Hours/week	12

Each column has a fixed index position in the dataset. We loop through every record using a for loop. Specific column values are appended into new lists. This separates the dataset into individual attribute lists.

Problem 4: Remove all records that contain missing values

Code:

```
cleaned_data = []
for row in dataset:
if "?" not in row:
cleaned_data.append(row)
print("Cleaned records:", len(cleaned_data))
```

Output:

```
Original records: 32562
Cleaned records: 30163
```

Explanation: Missing values in this dataset are represented using "?". We iterate through every record in the dataset. If a row does not contain "?", it is added to a new list. This creates a cleaned dataset without incomplete records.

Problem 5: Create: A subset containing the first 500 records and another one with the last 200 records. Concatenate these two lists into a new list.

Code:

```
first_500 = cleaned_data[:500]

last_200 = cleaned_data[-200:]

print("First 500:", len(first_500))

print("Last 200:", len(last_200))

combined_list = first_500 + last_200

print("Combined list size:", len(combined_list))
```

Output:

```
First 500: 500
Last 200: 200
Combined list size: 700
```

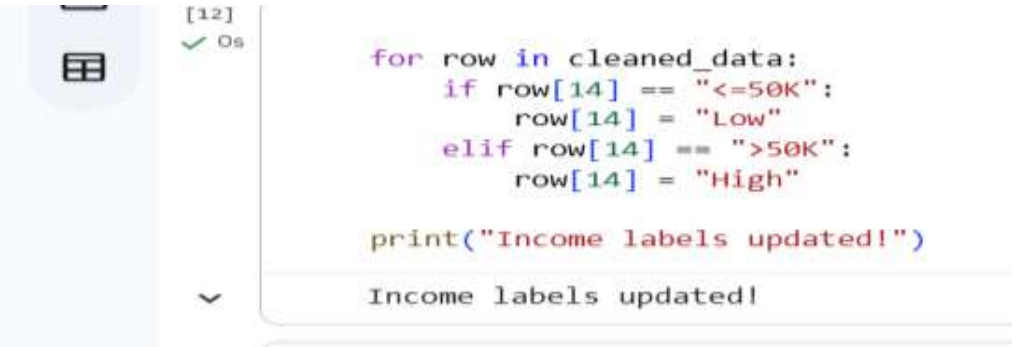
Explanation: List slicing allows extraction of specific portions of a list. The first 500 records are selected using slicing from index 0. The last 200 records are selected using negative slicing. This creates two smaller subsets from the cleaned dataset.

Problem 6: Replace all "<=50K" income labels with "Low" and ">50K" with "High".

Code:

```
for row in cleaned_data:
    if row[14] == "<=50K":
        row[14] = "Low"
    elif row[14] == ">50K":
        row[14] = "High"
print("Income labels updated!")
```

Output:



Explanation: The income column is located at index position 14. We loop through every cleaned record. Conditional statements are used to check income values. Old labels are replaced with new simplified labels.

Problem 7: Take the first 100 cleaned records and replicate them 5 times using *.

Code:

```
replicated_data = cleaned_data[:100] * 5
print("Replicated data size:", len(replicated_data))
```

Output:

```
[13]
✓ 0s
replicated_data = cleaned_data[:100] * 5

print("Replicated dataset size:", len(replicated_data))

Replicated dataset size: 500
```

Explanation: The * operator is used to repeat list elements. We first slice the first 100 cleaned records. Multiplying the list by 5 replicates it five times. This creates a larger dataset from a smaller subset.

Problem 8: Select any one record and assign its fields into multiple variables using list unpacking.

Code:

```
record = cleaned_data[0]

(age, workclass, final_weight, education, education_num,

 marital_status, occupation, relationship, race, sex,

 capital_gain, capital_loss, hours_per_week,

 native_country, income) = record

print("Unpacked record:")

print(age, education, occupation, income)
```

Output:

```
Unpacked record:
age education occupation income
```

Explanation: List unpacking assigns multiple values in one statement. Each variable corresponds to one field in the record. The number of variables must match the number of attributes. This improves readability compared to indexing repeatedly.

Problem 9: Ask the user to input an occupation and print all matching records using membership checking.

Code:

```
user_occ = input("Enter an occupation: ").strip()
print("\nMatching records:")
for row in cleaned_data:
    if user_occ in row:
        print(row)
```

Output:

Explanation: The input() function collects user input at runtime. We iterate through each cleaned record. Membership checking is done using the "in" operator. Matching records are printed if the occupation exists in the row.